

From SMURF to HI-SMURF: Scalable Multivariate Nonlinear Function Approximation via Compact Stochastic Architectures

Xincheng Feng , Wenyong Zhou , Taiqiang Wu , Zhengwu Liu , *Member, IEEE*, Meng Li ,
and Ngai Wong , *Senior Member, IEEE*

Abstract—Multivariate nonlinear functions are fundamental to many tasks in artificial intelligence (AI) and communication systems but are difficult to implement efficiently in hardware due to high computational complexity and resource demands, especially on edge and embedded devices. Existing stochastic computing (SC) approaches are mostly limited to univariate cases and do not scale well for high-dimensional approximation. This paper presents *SMURF* (Stochastic Multivariate Universal-Radix Finite-State Machine) and its enhanced variant *HI-SMURF* (High-Dimensional Stochastic Multivariate Universal-Radix Finite-State Machine) that uses tensor-train (TT) factorization to approximate high-dimensional functions without exponential state growth. *SMURF* analytically derives steady-state distributions and employs convex optimization for parameter synthesis, while *HI-SMURF* preserves accuracy under scalable input dimensionality by factorizing the parameter space. Both architectures use configurable state resolution and simple logic circuits driven by stochastic bitstreams. We evaluate *SMURF* and *HI-SMURF* on low-density parity-check (LDPC) decoding, image classification, image reconstruction, and high-dimensional nonlinear benchmarks against baselines including Taylor-series expansions, look-up table (LUT) methods, Bernstein polynomial SC, tensor-factorization methods, and quantized multilayer perceptrons (MLPs). Results show that our methods maintain comparable accuracy while dramatically reducing hardware area, power, and area-delay product (ADP), demonstrating superior scalability and efficiency for resource-constrained systems.

Index Terms—Nonlinear functions, stochastic computing, multivariate, high hardware efficiency.

Received 29 October 2025; revised 8 March 2026; accepted 25 April 2026. Date of publication 29 April 2026; date of current version 11 June 2026. This work was supported in part by the Theme-based Research Scheme (TRS) Project T45-701/22-R, in part by the National Natural Science Foundation of China under Grant 62404187, and in part by the ACCESS – AI Chip Center for Emerging Smart Systems, sponsored by InnoHK funding, Hong Kong SAR. Recommended for acceptance by K. Chakraborty. (*Xincheng Feng and Wenyong Zhou contributed equally to this work.*) (*Corresponding authors: Meng Li; Ngai Wong.*)

Xincheng Feng, Wenyong Zhou, Taiqiang Wu, Zhengwu Liu, and Ngai Wong are with the Department of Electrical and Computer Engineering, The University of Hong Kong, Hong Kong SAR, China (e-mail: nwong@eee.hku.hk).

Meng Li is with the Institute for Artificial Intelligence and School of Integrated Circuits, Peking University, Beijing 100871, China (e-mail: meng.li@pku.edu.cn).

Digital Object Identifier 10.1109/TC.2026.3688666

I. INTRODUCTION

MULTIVARIATE nonlinear functions are fundamental to a wide variety of computing tasks such as signal processing [1], [2], communications [3], [4], and artificial intelligence (AI) accelerators [5], [6], [7]. They enable accurate modeling of variable interactions that are critical for tasks like modulation and coding [8], [9], softmax and attention in neural networks [10], [11], and manifold-based transformations in data analytics [12], [13]. As embedded AI and edge computing workloads grow, the efficient hardware realization of these operators has become a central challenge [14], [15]. Such functions are often non-separable and require costly exponentiation, products, or complex activation profiles, which results in significant silicon area, power, and latency overheads under tight resource budgets. The rapid growth of deep learning [16], [17], vision transformers (ViTs) [18], [19], graph neural networks [20], and advanced signal processing pipelines has dramatically increased the volume of nonlinear computations performed in both training and inference. Modern AI models rely heavily on attention mechanisms [21], [22], normalization layers, and sophisticated activation functions, making the demand for efficient nonlinear operators especially acute in resource-constrained edge devices and large-scale data centers.

Traditional approximation strategies include Taylor-series expansions [23], coordinate rotation digital computer (CORDIC) algorithms [24], and look-up table (LUT) interpolation [25]. Despite their popularity, each has fundamental limitations: Taylor expansions often require high-order terms for complex or multivariate functions, increasing arithmetic depth and pipeline stages; CORDIC relies on iterative rotations, hindering efficient pipelining and low-latency design; and LUT interpolation scales exponentially in storage with dimensionality and resolution, becoming impractical for multivariate settings. These drawbacks motivate alternative approaches that better balance accuracy, scalability, and hardware efficiency.

Recent approximate computing methods include piecewise-linear arithmetic units [26], hybrid logic-table ensembles [27], Chebyshev polynomial evaluators [28], and analog or memristor based synthesizers [29]. Although they offer improved trade-offs in certain regimes, many are fixed-function,

hard to reconfigure, or rely on floating-point support and non-trivial control logic, and they are not intended for general-purpose multivariate nonlinear approximation, limiting their use in reconfigurable accelerators and edge devices.

Stochastic computing (SC) has emerged as a promising paradigm for lightweight and noise tolerant arithmetic [30]. By encoding real values as unary bitstreams and processing them through simple logic gates such as AND, XOR, and MUX, SC enables ultra low cost computation. More recently, finite state machine (FSM) based SC architectures have extended these benefits to function approximation [31], [32], mapping bitstream statistics to state transitions to approximate functions such as $\exp(x)$, $\tanh(x)$, and $\text{ReLU}(x)$. In parallel, Bernstein polynomial based SC realizes nonlinear mappings through stochastic evaluations of polynomial bases [33]. However, Bernstein approaches typically require long bitstreams to achieve high precision and become inefficient in high dimensional settings due to growth in required basis evaluations, which limits their practicality beyond univariate or low dimensional cases.

To address these challenges, we introduce the SMURF (*Stochastic Multivariate Universal-Radix FSM*) framework, which establishes a stochastic finite-state paradigm for approximating multivariate nonlinear functions in a compact and analytically tractable manner. SMURF models nonlinear mappings through probabilistic state transitions rather than explicit arithmetic operations, achieving substantially higher hardware efficiency than Taylor, LUT, Bernstein, or CORDIC baselines. By formulating nonlinear approximation in the probabilistic domain, SMURF enables flexible and hardware-friendly computation that inherently tolerates noise and quantization effects.

Building on this foundation, we present HI-SMURF (*High-Dimensional SMURF*), a generalized framework that extends SMURF to high-dimensional nonlinear functions. HI-SMURF incorporates tensor-based function factorization, allowing complex multivariate mappings to be expressed as combinations of lower-order components. Through tensor training (TT), it effectively captures cross-dimensional dependencies while maintaining tractable state complexity and stochastic precision. This formulation provides a compact and scalable realization of high-dimensional nonlinear operators without exponential growth in area or power. Together, SMURF and HI-SMURF form a unified SC framework that connects theoretical modeling with efficient hardware implementation for nonlinear function approximation.

The main contributions of this work are summarized as follows:

- SMURF introduces a probabilistic finite-state formulation that represents nonlinear functions through universal-radix transitions, providing a principled and compact alternative to arithmetic-based computation.
- HI-SMURF extends this formulation to high-dimensional domains by integrating TT with stochastic state modeling, enabling efficient synthesis of multivariate nonlinear operators with controllable accuracy and resource cost.
- Extensive software and hardware evaluations demonstrate that the proposed architectures achieve substantial

improvements in area, power, and area-delay product (ADP) efficiency while maintaining comparable accuracy to traditional approximation methods.

- The proposed frameworks are applied to representative computing tasks, including low-density parity-check (LDPC) decoding and vision transformer (ViT) activation modeling, confirming their scalability and practical relevance across communication and AI hardware.

The remainder of this paper is organized as follows. Section II reviews the preliminaries, including SC fundamentals, sampling gates, FSM-based nonlinear modeling, and tensor-train decomposition. Section III presents the SMURF and HI-SMURF architectures, detailing their theoretical basis and tensor-based formulation. Section IV provides experimental and hardware results. Section V concludes the paper.

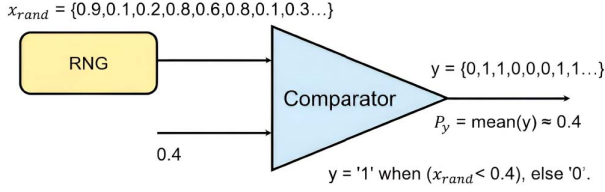
II. PRELIMINARIES

A. Stochastic Computing (SC)

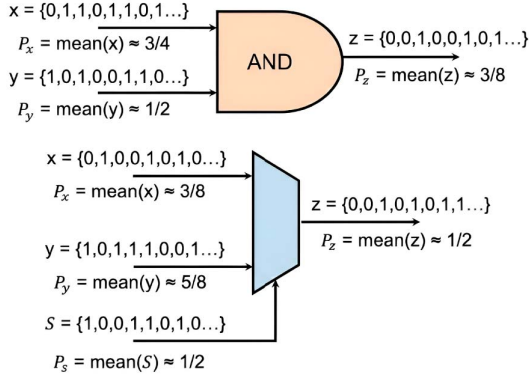
SC is an innovative computational paradigm that differs fundamentally from conventional fixed-point binary computing [30]. In SC, numbers are represented as random bitstreams, where the statistical proportion of 1's in the sequence corresponds to the value being represented. These sequences are generated by stochastic bitstream generators, which produce random patterns of 0's and 1's based on the target value. To decode a stochastic number, the proportion of 1's in the stream is calculated by counting the number of 1's using a binary counter and determining the stream's average, as illustrated in Fig. 1. This unique representation enables SC to perform complex computations using exceptionally simple hardware structures.

As depicted in Fig. 1, SC realizes arithmetic operations with extremely lightweight logic. Fig. 1(a) first illustrates a typical stochastic number generator (SNG), which converts a real-valued input into a bitstream by comparing it against a random sequence produced by an RNG through a binary comparator. Fig. 1(b) then shows two canonical SC arithmetic primitives. For multiplication, two independent stochastic bitstreams x and y with means P_x and P_y are fed to a single AND gate, producing an output stream z whose mean satisfies $P_z = P_x P_y$. For addition, a 2-to-1 multiplexer (MUX) selects between x and y using a scaling bitstream s with $P_s = 1/2$, yielding $P_z = \frac{1}{2}P_x + \frac{1}{2}P_y = (P_x + P_y)/2$. If an unscaled sum is needed, the factor of $1/2$ can be compensated by a left shift in conventional binary post-processing. In contrast to fixed-point or floating-point arithmetic that relies on dedicated multipliers/adders and nontrivial carry handling, these SC primitives avoid carry propagation and multi-stage bitwise logic, significantly reducing hardware area, power consumption, and design complexity.

Although SC is typically used for computing stochastic number (SN) values in $[0, 1]$, we can apply SC to compute any function by normalizing its input-output variables to $[0, 1]$. This normalization step can be done by a simple bijective linear transformation. After the SC computation, we can map the output value back to its original range by linear scaling. Moreover, in a convolution process, one can utilize SC for point-wise



(a) Stochastic number generator (SNG) consisting of a binary comparator and a random number generator (RNG).



(b) SC multiplication and addition primitives.

Fig. 1. Illustration of key components in the SC paradigm.

matrix multiplications (PwMMs) and apply SC to activation functions [34], resulting in significantly improved energy efficiency compared to fixed-point multiplications.

B. Random Sampling Gates

Random sampling gates [35] can be used to implement any function with known probability distributions. The logic gate used to map one probability distribution directly to another is called the θ -gate. The logic gate used to map one probability distribution to another under a condition is called the conditional probability table (CPT) gate.

θ -gate samples a random number set according to a certain probability to obtain a binary Bernoulli distribution. By comparing the value between a prescribed input value and a random entropy source, one obtains a binary (0, 1) output value. The gate can sample exactly from each given distribution and produce one sample per cycle, e.g., SNG in Fig. 1 is a simple θ -gate. As the sequence becomes sufficiently long, the output of a θ -gate approaches the set distribution. This is also equivalent to the Monte Carlo method. A θ -gate can also sample complex probability distributions such as the Sobol sequences. Henceforth, for convenience in further explanations, we refer to all gates that convert a full-precision value into an SN as θ -gates.

CPT-gate is a collection of θ -gates, together with a MUX to select one of the θ -gates as its output. This is equivalent to rolling a biased dice and choosing which dice value to take based on the input. In practice, we can adjust the distribution of the overall system output by modifying the threshold values of the θ -gates. The control of the MUX often involves the use of random devices such as an FSM whose state transitions follow specific probabilistic rules.

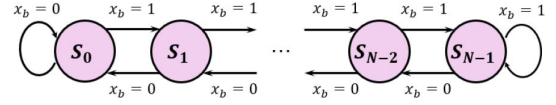


Fig. 2. The architecture of a chained N -state FSM where x_b denotes the current bitstream (binary) value from a θ -gate.

C. FSM-Based Nonlinear Function Generator

Brown and Card [31] showed that an N -state FSM can realize a desired steady-state output distribution by assigning each state output to 0 or 1. Fig. 2 illustrates a chained FSM: the state moves right (or saturates at the rightmost) when $x_b = 1$, and moves left otherwise. For an even- N FSM, if states 1 to $N/2$ output 0 and states $(N/2) + 1$ to N output 1, the resulting stochastic mapping follows a tanh-like characteristic. Let x and y denote the input/output SNs; then

$$P_y \approx \frac{e^{\frac{N}{2}P_x} - e^{-\frac{N}{2}P_x}}{e^{\frac{N}{2}P_x} + e^{-\frac{N}{2}P_x}}. \quad (1)$$

If we reverse the state outputs (states 1 to $N/2$ output 1 and states $(N/2) + 1$ to N output 0) and reorder state indices as $0, 1, \dots, N - G - 1, N - G, \dots, N - 2, N - 1$ with an integer G satisfying $1 \leq G \leq N - 1$, the input-output mapping becomes exponential:

$$P_y \approx e^{-2Gx}. \quad (2)$$

These examples indicate that the state-output assignment and index ordering can reshape the stochastic transfer function, enabling flexible nonlinear mappings beyond exp and tanh.

More generally, an N -state chain with transition probabilities in $[0, 1]$ is traversable and aperiodic, and its evolution can be viewed as a steady-state Markov process [36]. Fig. 3 reports the steady-state occupancies of 2-, 3-, 4-, and 5-state FSMs from Monte Carlo simulations. Specifically, for each input right-transition probability swept from 0 to 1 with a step of 0.01, we ran 256 transitions per trial and repeated 10,000 trials, recording the state counts. As P_x increases, the leftmost state's occupancy decreases from 1 to 0 while the rightmost increases symmetrically from 0 to 1; intermediate states form distinct "humps," which provide smooth interpolation between the extremes. This non-uniform steady-state structure explains why edge states cover broader output ranges (coarser changes) whereas middle states refine the output with higher granularity, which is crucial for accurate nonlinear approximation over $[0, 1]$.

We further observe diminishing accuracy gains when increasing N beyond a moderate size. Across all tested cases, a 4-state FSM chain provides a favorable precision-complexity trade-off, offering sufficient flexibility for diverse nonlinear functions with low hardware overhead.

D. Tensor Train (TT) Decomposition

A d -way tensor is denoted by $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$, where d is the order, and n_k is the size of the k -th mode. Directly storing $\mathcal{A}$ requires $\prod_{k=1}^d n_k$ parameters, which grows exponentially in d . TT decomposition [37] is a low-rank factorization that

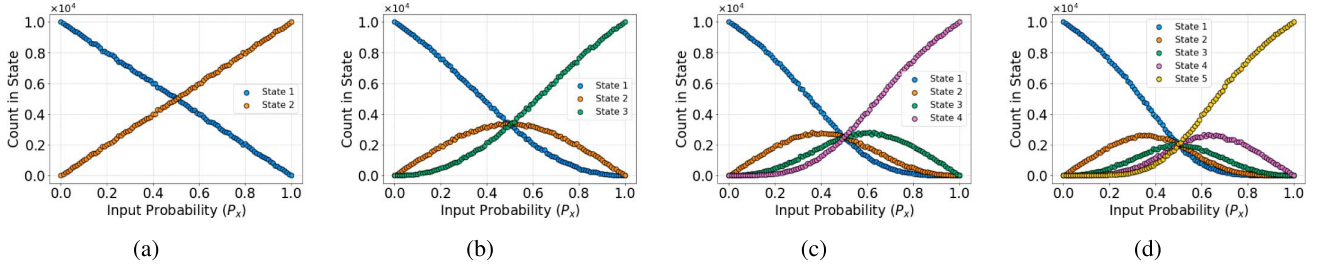


Fig. 3. (a)-(d) The steady-state counts of points in each state for 2-, 3-, 4-, and 5-state FSMs under Monte Carlo experiments.

represents $\mathcal{A}$ by a chain of smaller three-way tensors called *TT-cores*. Specifically,

$$\mathcal{A}(i_1, \dots, i_d) \approx G^{(1)}(1, i_1, r_1) G^{(2)}(r_1, i_2, r_2) \dots G^{(d)}(r_{d-1}, i_d, 1), \quad (3)$$

where $i_k \in \{1, \dots, n_k\}$ indexes along the k -th mode, $G^{(k)} \in \mathbb{R}^{r_{k-1} \times n_k \times r_k}$ is the k -th TT-core, and $r_0 = r_d = 1$ while $r_1, \dots, r_{d-1}$ are called the *TT-ranks*, controlling the expressive power. Under TT, the number of trainable parameters becomes

$$\sum_{k=1}^d r_{k-1} n_k r_k, \quad (4)$$

which scales linearly with d when all r_k are bounded, alleviating the exponential storage growth. TT decomposition is therefore widely used in compressing high-dimensional functions and operators.

III. SMURF AND HI-SMURF METHODOLOGY

This section presents the stochastic FSM architecture and the steady-state derivation for SMURF and its high-dimensional extension HI-SMURF. We first introduce the *bivariate* ($M=2$) case and then generalize to the *multivariate* setting. Throughout, M denotes the number of input variables, each encoded as an SN $P_{x_m} \in [0, 1]$; the m -th FSM has N states indexed by $i_m \in \{0, \dots, N-1\}$; the right/left transition probabilities of the m -th FSM are P_{x_m} and $\overline{P_{x_m}} = 1 - P_{x_m}$, respectively; we also use the transit ratio $t_{x_m} = P_{x_m} / \overline{P_{x_m}}$. The bitstream output is y_b , whose time average P_y approximates the target function $T(\cdot)$.

As illustrated in Fig. 4, each real-valued input P_{x_m} is represented by a Bernoulli bitstream $x_{b_m}(t)$. At every clock, $x_{b_m}(t)=1$ triggers $S_{i_m}^{(m)} \rightarrow S_{i_m+1}^{(m)}$ and $x_{b_m}(t)=0$ triggers $S_{i_m}^{(m)} \rightarrow S_{i_m-1}^{(m)}$ with boundary reflections. The M FSM states concatenate into a universal-radix codeword $\mathbf{s} = [i_M, \dots, i_1]$. A CPT stores one parameter $w_s \in [0, 1]$ per codeword; a MUX selects w_s to emit $y_b(t) \sim \text{Bernoulli}(w_s)$. Hence the time average $P_y = \frac{1}{L} \sum_{t=1}^L y_b(t)$ equals $\sum_{\mathbf{s}} P_{\mathbf{s}} w_{\mathbf{s}}$ in expectation, with variance $\mathcal{O}(1/L)$.

A. Bivariate SMURF ($M=2$)

Let $\mathbf{s} = [i_2, i_1]$ and $P_{\mathbf{s}} = P_{[i_2, i_1]}$ denote the joint steady-state probability of being in states $S_{i_2}^{(2)}$ and $S_{i_1}^{(1)}$. At steady state,

$$P_{[i_2, i_1+1]} \overline{P_{x_1}} = P_{[i_2, i_1]} P_{x_1}, \quad (5a)$$

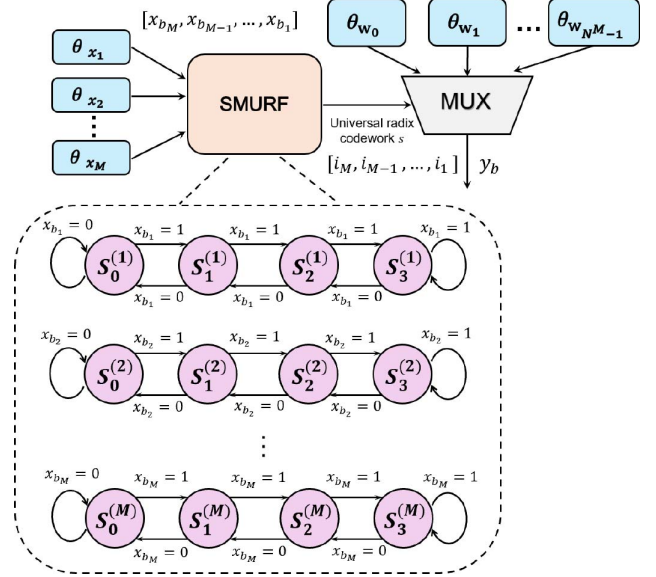


Fig. 4. SMURF architecture. M stochastic inputs drive M independent N -state FSMs; their concatenated states form a codeword $\mathbf{s}$ that selects CPT entry $w_{\mathbf{s}}$ via a MUX to emit y_b . blue boxes θ_{x_m} are input gates and θ_{w_j} are table parameters.

$$P_{[i_2+1, i_1]} \overline{P_{x_2}} = P_{[i_2, i_1]} P_{x_2}. \quad (5b)$$

Defining $t_{x_1} = P_{x_1} / \overline{P_{x_1}}$ and $t_{x_2} = P_{x_2} / \overline{P_{x_2}}$, we obtain $P_{[i_2, i_1+1]} = P_{[i_2, i_1]} t_{x_1}$ and $P_{[i_2+1, i_1]} = P_{[i_2, i_1]} t_{x_2}$, hence

$$P_{[i_2, i_1]} = \frac{t_{x_2} t_{x_1}}{\left(\sum_{u=0}^{N-1} t_{x_2}^u \right) \left(\sum_{v=0}^{N-1} t_{x_1}^v \right)}. \quad (6)$$

Let $w_s \in [0, 1]$ denote the selection probability of the θ -gate associated with codeword $\mathbf{s}$ (i.e., the CPT parameter). The expected output is the mixture

$$P_y = \sum_{i_2=0}^{N-1} \sum_{i_1=0}^{N-1} P_{[i_2, i_1]} w_{[i_2, i_1]}. \quad (7)$$

Given a bivariate target function $T(P_{x_1}, P_{x_2})$, SMURF learns $\{w_s\}$ by minimizing the mean-squared error

$$\epsilon = \int_0^1 \int_0^1 (T(P_{x_1}, P_{x_2}) - P_y)^2 dP_{x_1} dP_{x_2}. \quad (8)$$

Expanding and collecting terms yields a convex quadratic form in the vector $\mathbf{b} = [w_{[0,0]}, \dots, w_{[N-1,N-1]}]^T$:

$$\varphi = \mathbf{b}^T \mathbf{H} \mathbf{b} + 2 \mathbf{c}^T \mathbf{b}, \quad (9)$$

where the entries of $\mathbf{H}$ and $\mathbf{c}$ are analytic integrals over $[0, 1]^2$ involving $P_{[i_2, i_1]}$ from (6) and $T(\cdot)$ (omitted for brevity).

B. Multivariate SMURF ($M > 2$)

For $\mathbf{s} = [i_M, \dots, i_1]$ with all FSMs having N states, repeating the argument of (5) leads to

$$P_{\mathbf{s}} = \frac{\prod_{m=1}^M t_{x_m}^{i_m}}{\prod_{m=1}^M \left(\sum_{u=0}^{N-1} t_{x_m}^u \right)}. \quad (10)$$

Let $\mathcal{W} \in [0, 1]^{N \times \dots \times N}$ (order- M) collect all CPT parameters $w_{\mathbf{s}}$. The expected output is

$$P_y = \sum_{i_M=0}^{N-1} \dots \sum_{i_1=0}^{N-1} P_{[i_M, \dots, i_1]} w_{[i_M, \dots, i_1]}, \quad (11)$$

and the population mean squared error (MSE) objective generalizes (8) to M -fold integration over $[0, 1]^M$.

C. HI-SMURF: Tensorized Parameterization for High Dimensions

The bottleneck of SMURF is the size of $\mathcal{W}$, which scales as N^M . HI-SMURF replaces the explicit table by a TT factorization that preserves the same stochastic pipeline while removing exponential growth. Compared to SMURF's parameter space, this factorization only introduces a few additional matrix multiplications, which can be realized in the stochastic domain using simple AND gates, keeping hardware overheads minimal. Each TT core $G^{(m)} \in \mathbb{R}^{r_{m-1} \times N \times r_m}$ maps index i_m to a slice, with $r_0 = r_M = 1$ and $r_1 = \dots = r_{M-1} = r$. The TT chain product (Fig. 5)

$$\tilde{w}_{[i_M, \dots, i_1]} = G^{(1)}[i_1] G^{(2)}[i_2] \dots G^{(M)}[i_M] \quad (12)$$

produces the CPT value, replacing $w_{\mathbf{s}}$ in (11):

$$P_y = \sum_{\mathbf{s}} P_{\mathbf{s}} \tilde{w}_{\mathbf{s}}. \quad (13)$$

Min-max calibration keeps compatibility with Bernoulli emission. Apart from this tensor-train parameter storage, HI-SMURF is otherwise identical to SMURF in algorithmic principle and stochastic processing.

D. Parameter Counts and Switching Criterion

For SMURF (scalar output): $|\mathcal{W}| = N^M$. For HI-SMURF (scalar, TT rank r), the parameter count is

$$|\{G^{(m)}\}| = (1 \cdot N \cdot r) + (M-2)(r \cdot N \cdot r) + (r \cdot N \cdot 1) = 2Nr + (M-2)Nr^2. \quad (14)$$

For K outputs (the last core is $G^{(M)} \in \mathbb{R}^{r \times N \times K}$),

$$|\{G^{(m)}\}| = Nr + (M-2)Nr^2 + NrK, \quad (15)$$

Algorithm 1 SMURF: CPT-based FSM Approximation

Require: Input dimension M , FSM states N , target function $T(\cdot)$, training size S , bitstream length L
Initialize CPT tensor $\mathcal{W} \in [0, 1]^{N \times \dots \times N}$
for $s = 1$ to S **do**
 Sample stochastic inputs $P_{x_1}, \dots, P_{x_M}$
 Compute steady-state weight $P_{\mathbf{s}}$ via Eq. (6)
 Evaluate $t_s \leftarrow T(P_{x_1}, \dots, P_{x_M})$
Solve Eq. (9) to obtain parameters $w_{\mathbf{s}}$ in $\mathcal{W}$
function INFERENCE($P_{x_1}, \dots, P_{x_M}$)
 $P_y \leftarrow 0$
 for $t = 1$ to L **do**
 Update FSMs $\Rightarrow$ obtain codeword $\mathbf{s}(t)$
 Emit $y_b(t)$ using LUT entry $w_{\mathbf{s}(t)}$
 $P_y \leftarrow P_y + y_b(t)$
 return P_y/L

$$|\mathcal{W}|_{\text{SMURF}} = K N^M. \quad (16)$$

Therefore, we prefer HI-SMURF when

$$N^M \geq 2Nr + (M-2)Nr^2. \quad (17)$$

Both SMURF and HI-SMURF minimize

$$\epsilon = \int_{[0,1]^M} (T(P_{x_1}, \dots, P_{x_M}) - P_y)^2 dP_{x_1} \dots dP_{x_M}, \quad (18)$$

where P_y equals (7) (SMURF) or (13) (HI-SMURF). Monte Carlo sampling over $[0, 1]^M$ is used in practice. SMURF solves the convex quadratic (9) (with optional box constraints $w_{\mathbf{s}} \in [0, 1]$). HI-SMURF optimizes $\{G^{(m)}\}$ by adaptive moment estimation (Adam); useful tricks include ℓ_2 regularization, learning rate (LR) warmup, cosine decay, and occasional core rescaling to avoid saturation.

Training on expected P_y (no sampling noise) exposes pure approximation capacity. Including Bernoulli emission and finite window L makes the loss inherit variance $\mathcal{O}(1/L)$; larger L reduces variance but increases latency.

Given $\{P_{x_m}\}$, hardware (i) generates bitstreams $x_{b_m}(t)$, (ii) updates the M FSMs to obtain $\mathbf{s}(t)$, (iii) performs a CPT lookup (SMURF) or TT chain (HI-SMURF, cost $\mathcal{O}(Mr^2)$) to obtain $w_{\mathbf{s}(t)}$ or $\tilde{w}_{\mathbf{s}(t)}$, and (iv) averages $y_b(t)$ over L cycles. SMURF favors minimal compute and larger memory; HI-SMURF trades modest compute for linear-in- M storage.

Note. Algorithm 2 differs from Algorithm 1 only in parameter storage: it replaces the dense CPT with a tensor-train factorization, while the stochastic pipeline, FSM updates, and inference remain unchanged.

E. Discussion

SMURF is a stochastic, table-parameterized approximator with analytic mixing weights $P_{\mathbf{s}}$ and variance scaling $\mathcal{O}(1/L)$. HI-SMURF preserves the same Markov front-end and mixing law while tensorizing the CPT via TT cores, yielding linear-in- M storage at fixed rank r and $\mathcal{O}(Mr^2)$ per-cycle compute.

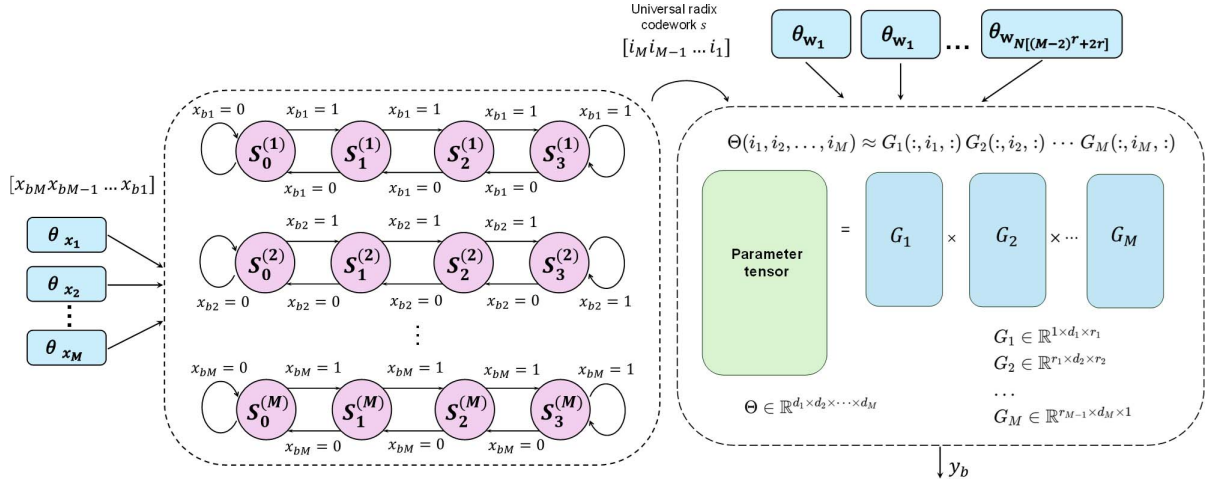


Fig. 5. **HI-SMURF Architecture.** Shares the stochastic front-end and codeword s with SMURF, but computes $\tilde{w}_s$ by contracting TT slices $G^{(1)}[i_1] \dots G^{(M)}[i_M]$ from rank- r cores. The green tensor Θ represents the implicit table realized by the TT factorization.

Algorithm 2 HI-SMURF: Tensor-Train based High-Dimensional SMURF

Require: Input dimension M , FSM states N , TT rank r , target function $T(\cdot)$, training size S , bitstream length L
Initialize TT cores $G^{(m)} \in \mathbb{R}^{r_{m-1} \times N \times r_m}$ with $r_0 = r_M = 1$
for $s = 1$ to S **do**
 Sample stochastic inputs $P_{x_1}, \dots, P_{x_M}$
 Compute P_s as in Eq. (6)
 Compute $\tilde{w}_s = \prod_{m=1}^M G^{(m)}[i_m]$ via Eq. (12)
 Form $P_y = \sum_s P_s \tilde{w}_s$ by Eq. (13)
 Compute loss $(T(P_{x_1}, \dots, P_{x_M}) - P_y)^2$
Update $\{G^{(m)}\}$ by Adam
function INFERENCE($P_{x_1}, \dots, P_{x_M}$)
 $P_y \leftarrow 0$
 for $t = 1$ to L **do**
 Update FSMs $\Rightarrow$ codeword $s(t)$
 Compute $\tilde{w}_{s(t)}$ from TT cores
 Emit $y_b(t)$
 $P_y \leftarrow P_y + y_b(t)$
 return P_y/L

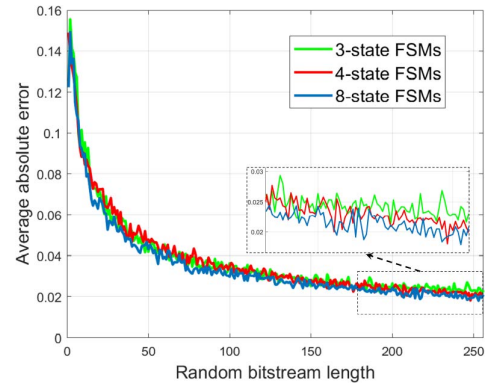


Fig. 6. SMURF approximation errors in a 3-variate softmax function with 3/4/8-state FSMs w.r.t. bitstream lengths.

implemented in Semiconductor Manufacturing International Corp. (SMIC) 65 nm technology and operated at 400 MHz, SMURF and HI-SMURF deliver substantially lower area, power, and area-delay product (ADP) than conventional baselines.

A. Performance of SMURF in Generic Nonlinearities

Standard fixed-point representation is employed for θ -gate inputs, whose quantization error is negligible compared to other sources of errors, such as the Markov process uncertainties and random bitstream averaging.

We first test the computing accuracy of SMURF in a 3-variate softmax function using 3-state, 4-state and 8-state FSMs for each input variable. The average absolute error with respect to the bitstream length is shown in Fig. 6, showing the fast decay and convergence of the average absolute errors. The average absolute errors are all around 0.15 near zero, and decrease to 0.02 at a bitstream length of 256. On the other hand, increasing the number of states in an FSM results in only small gains in output precision ($\leq 10^{-2}$). The main causes of the error are the stochastic process (dynamic stability) of the FSMs, and the instability introduced by the θ -gates and CPT-gates (namely, fluctuations in mean computation), both arising from the

The choice between them is governed by parameter budget and target dimensionality per the switching rules above.

IV. EXPERIMENTAL RESULTS

We evaluate SMURF and HI-SMURF across generic nonlinear operators, application tasks, and hardware benchmarks. SMURF exhibits clear $1/L$ -like error decay and outperforms LUT, Taylor, CORDIC, and Bernstein under comparable precision, while HI-SMURF scales to $M=16$ variables with accuracy competitive to a quantized multilayer perceptron (MLP). In applications, SMURF improves LDPC decoding bit error rate (BER), achieves competitive image reconstruction compared with traditional implicit neural representations (INRs), and HI-SMURF yields consistent gains when replacing softmax in a Tiny ViT. On field-programmable gate array (FPGA)

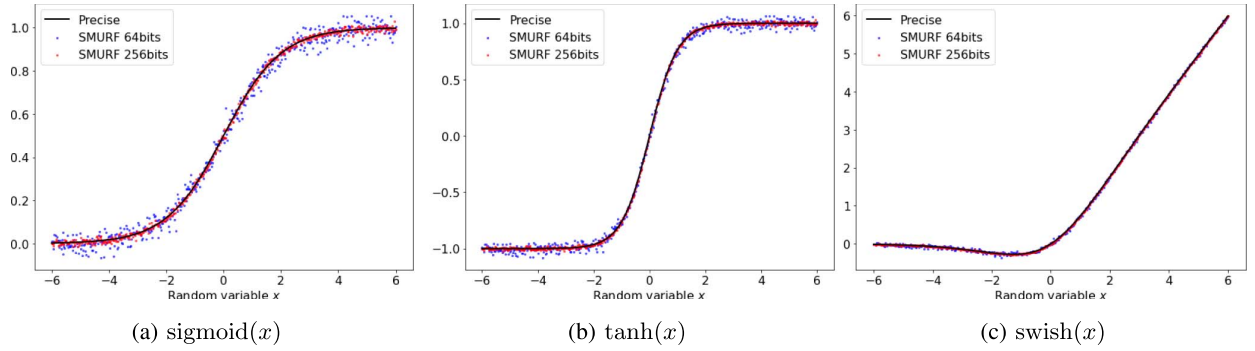


Fig. 7. The performance of SMURF approximating univariate functions with the bitstream length of 64 and 256.

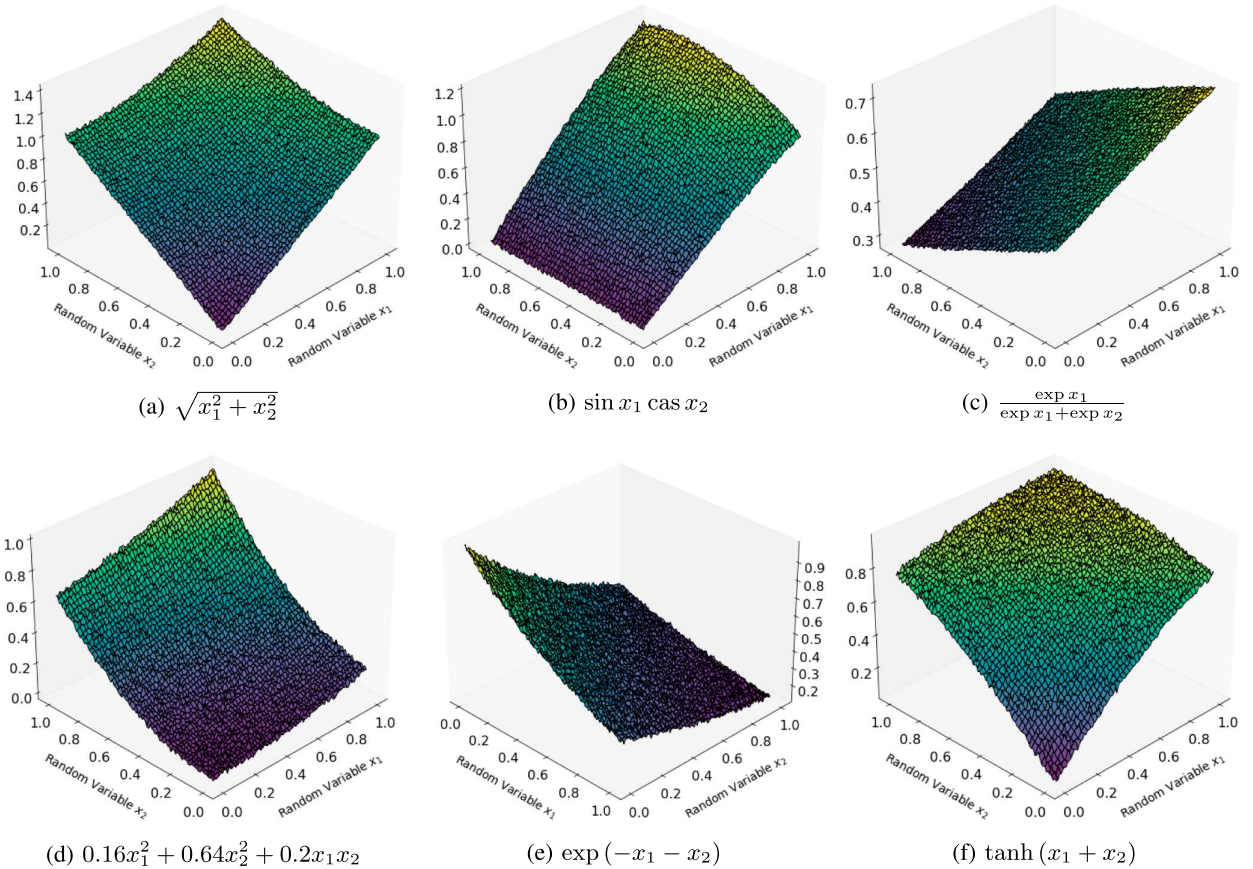


Fig. 8. The performance of SMURF approximating bivariate functions with a bitstream length of 256. (note: $\text{cas}(x) = \sin(x) + \cos(x)$).

finite-length random bitstreams. That said, in practical applications we often do not require a very high precision, e.g., prior work has shown CNNs can tolerate the relatively low accuracy of SC [34]. Balancing hardware complexity and accuracy, we fix the bitstream length at 64, whose average absolute error is around 0.04 in this example.

Next, we approximate the common neural network activation functions, sigmoid, tanh, and swish, using SMURF with bitstream lengths of 64 and 256. Fig. 7 depicts the approximation results. It can be seen that SMURF achieves a good approximation in all functions at the bitstream length of 64, and can highly restore these two nonlinearities at a length of 256. The error at even 64 bits is adequately small for neural network inferences.

Fig. 8 shows the performance of SMURF approximating various bivariate nonlinear functions with an input bitstream length of 256. These figures validate that SMURF exhibits generally good approximation accuracy in multivariate nonlinear functions without excessively long bitstreams.

Table I reports the mean absolute error (MAE) and root mean square error (RMSE) of SMURF when approximating a wide range of bivariate and multivariate nonlinear functions at bitstream lengths of 64, 256, and 1024. Across all tested functions, both MAE and RMSE decrease steadily as the bitstream length increases, with the average error converging near $\text{MAE} \approx 3.2 \times 10^{-3}$ and $\text{RMSE} \approx 3.5 \times 10^{-3}$ at $L = 1024$, though individual functions show noticeable variation around these averages. For

TABLE I
THE PERFORMANCE OF SMURF IMPLEMENTING VARIOUS BIVARIATE AND MULTIVARIATE NONLINEAR FUNCTIONS (MAE / RMSE)

Functions	MAE@L = 64	RMSE@L = 64	MAE@L = 256	RMSE@L = 256	MAE@L = 1024	RMSE@L = 1024
$\sqrt{x_1^2 + x_2^2}$	2.13×10^{-2}	2.12×10^{-2}	1.02×10^{-2}	1.06×10^{-2}	2.95×10^{-3}	3.21×10^{-3}
$\sin x_1 \cos x_2$	2.31×10^{-2}	2.31×10^{-2}	1.11×10^{-2}	1.15×10^{-2}	3.64×10^{-3}	3.85×10^{-3}
$\frac{\exp x_1}{\exp x_1 + \exp x_2}$	1.89×10^{-2}	2.05×10^{-2}	1.01×10^{-2}	9.99×10^{-3}	2.88×10^{-3}	3.14×10^{-3}
$0.16x_1^2 + 0.64x_2^2 + 0.2x_1x_2$	2.03×10^{-2}	2.20×10^{-2}	1.06×10^{-2}	1.11×10^{-2}	3.47×10^{-3}	3.70×10^{-3}
$\exp(-x_1 - x_2)$	1.87×10^{-2}	1.95×10^{-2}	9.64×10^{-3}	9.84×10^{-3}	3.12×10^{-3}	3.36×10^{-3}
$\log(x_1 + x_2)$	2.38×10^{-2}	2.37×10^{-2}	1.13×10^{-2}	1.17×10^{-2}	3.81×10^{-3}	4.07×10^{-3}
$\tanh(x_1 + x_2)$	1.93×10^{-2}	1.98×10^{-2}	9.97×10^{-3}	9.91×10^{-3}	3.22×10^{-3}	3.50×10^{-3}
$\frac{x_1}{x_1 + x_2}$	2.11×10^{-2}	2.21×10^{-2}	1.11×10^{-2}	1.15×10^{-2}	3.40×10^{-3}	3.63×10^{-3}
$1 + \exp(-(0.15x_1 + 0.37x_2 + 0.23x_3 + 0.54))$	2.43×10^{-2}	2.46×10^{-2}	1.26×10^{-2}	1.26×10^{-2}	3.99×10^{-3}	4.25×10^{-3}
$\sqrt{\sum_{i=1}^4 (x_1^i - x_2^i)^2}$	2.29×10^{-2}	2.34×10^{-2}	1.16×10^{-2}	1.21×10^{-2}	3.33×10^{-3}	3.57×10^{-3}

example, for the Euclidean norm function, the RMSE drops from 2.12×10^{-2} at $L = 64$ to 1.06×10^{-2} at $L = 256$, and further to 3.21×10^{-3} at $L = 1024$, while the corresponding MAE decreases from 2.13×10^{-2} to 1.02×10^{-2} and then to 2.95×10^{-3} . Other functions, such as the sine-cosine combination and logistic cases, exhibit slightly higher values (RMSE up to 4.25×10^{-3}) at $L = 1024$, reflecting natural fluctuations in approximation difficulty. This trend follows the expected $1/L$ -like error reduction, confirming SMURF's accuracy benefits from longer bitstreams.

In addition, SMURF demonstrates robust generalization across diverse nonlinearities. It maintains consistent accuracy not only for elementary trigonometric forms but also for more complex multivariate cases including logistic and softmax-like functions. This versatility, combined with steadily decreasing errors, highlights SMURF's suitability for practical nonlinear computations involving both bivariate and multivariate functions.

To evaluate SMURF's approximation capability under realistic low-precision conditions, we compared it with several classical approximation methods, including LUT, Taylor series, CORDIC, and Bernstein, using the average errors summarized in Table II. The simulations were conducted under two representative precision settings: for LUT, Taylor, and CORDIC, 8-bit and 16-bit fixed-point quantization were used, while SMURF and Bernstein were evaluated with bitstream lengths of 256 and 1024. As shown in Table II, SMURF achieves substantially lower approximation errors (MAE = 1.10×10^{-2} , RMSE = 1.15×10^{-2} at $L = 256$, and further reduced to MAE = 3.20×10^{-3} , RMSE = 3.50×10^{-3} at $L = 1024$) compared to all baselines. These results demonstrate SMURF's superior accuracy for nonlinear computations under comparable resource conditions and highlight its ability to benefit significantly from longer bitstreams.

B. The Performance of HI-SMURF in Generic Nonlinearities

Table III reports MAE and RMSE for HI-SMURF on representative 16-variable nonlinear functions. As the bitstream length increases from $L = 256$ to $L = 1024$, errors decrease

TABLE II
AVERAGE MAE/RMSE OF DIFFERENT METHODS ON REPRESENTATIVE NONLINEAR FUNCTIONS. SMURF/BERNSTEIN USE BITSTREAM LENGTHS 256 AND 1024; LUT, TAYLOR, AND CORDIC USE 8- AND 16-BIT QUANTIZATION

Method	Lower Precision		Higher Precision	
	MAE	RMSE	MAE	RMSE
SMURF (L=256 / 1024)	0.0110	0.0115	0.0032	0.0035
Bernstein (L=256 / 1024)	0.0516	0.1716	0.0152	0.0480
LUT (8-bit / 16-bit)	0.0286	0.0341	0.0079	0.0098
Taylor (8-bit / 16-bit)	0.0293	0.0350	0.0082	0.0101
CORDIC (8-bit / 16-bit)	0.0293	0.0350	0.0075	0.0093

for all functions, but the convergence levels differ by functional class. For the softmax-type ratio and the Gaussian error linear unit (GELU)-sum, the errors reach the 3×10^{-3} to 4×10^{-3} range at $L = 1024$ (e.g., RMSE $\approx 4.10 \times 10^{-3}$ and $\approx 4.05 \times 10^{-3}$, respectively). The multiplicative sinusoid converges slightly higher at $\approx 4.39 \times 10^{-3}$, reflecting its stronger oscillations. In contrast, the 16-dimensional Gaussian radial function remains around the 10^{-2} scale even at $L = 1024$ (e.g., MAE $\approx 9.52 \times 10^{-3}$, RMSE $\approx 9.76 \times 10^{-3}$), since its narrow peak and heavy concentration near the center make stochastic estimation more variance-sensitive. Overall, HI-SMURF exhibits the expected $1/L$ -like decay, while the attainable error floor depends on the function's curvature and output dynamic range.

For comparison, we also trained a compact MLP consisting of two fully connected layers with 64 neurons each, ReLU activations, and a sigmoid output layer matching the target function. Both HI-SMURF and the MLP were trained on uniformly sampled points from $[0, 1]^{16}$ using the Adam optimizer with a learning rate of 10^{-3} for 600 epochs and the same mean-squared error objective. The MLP was subsequently evaluated under 8-bit and 16-bit post-training quantization to match the low- and high-precision scenarios of HI-SMURF at $L = 256$ and $L = 1024$, respectively.

TABLE III
COMPARISON OF HI-SMURF AND QUANTIZED MLP ON $M=16$ MULTIVARIATE FUNCTIONS UNDER LOW PRECISION ($L=256$ VS. 8-BIT) AND HIGH PRECISION ($L=1024$ VS. 16-BIT). LOWER VALUES INDICATE BETTER ACCURACY

Function	HI-SMURF ($L=256$)		MLP (8-bit)		HI-SMURF ($L=1024$)		MLP (16-bit)	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
$\frac{e^{x_1}}{\sum_{i=1}^{16} e^{x_i}}$	8.70×10^{-3}	9.38×10^{-3}	1.817×10^{-2}	2.236×10^{-2}	3.85×10^{-3}	4.10×10^{-3}	1.831×10^{-2}	2.256×10^{-2}
$\exp\left(-\sum_{i=1}^{16} (x_i - 0.5)^2\right)$	1.95×10^{-2}	2.53×10^{-2}	6.694×10^{-2}	8.414×10^{-2}	9.52×10^{-3}	9.76×10^{-3}	6.681×10^{-2}	8.399×10^{-2}
$\prod_{i=1}^{16} \sin(\pi x_i)$	9.12×10^{-3}	9.81×10^{-3}	2.351×10^{-2}	2.530×10^{-2}	4.08×10^{-3}	4.39×10^{-3}	2.352×10^{-2}	2.534×10^{-2}
$\sum_{i=1}^{16} \frac{x_i}{2} \left(1 + \operatorname{erf}\frac{x_i}{\sqrt{2}}\right)$ (GELU-sum)	8.40×10^{-3}	9.02×10^{-3}	2.785×10^{-2}	3.506×10^{-2}	3.76×10^{-3}	4.05×10^{-3}	2.792×10^{-2}	3.533×10^{-2}

TABLE IV
TEST MAE AND RMSE OF HI-SMURF ON THE $M=16$ SOFTMAX FUNCTION UNDER DIFFERENT TENSOR-TRAIN RANKS r

Rank r	Test MAE	Test RMSE
2	9.80×10^{-3}	1.04×10^{-2}
4	9.45×10^{-3}	9.89×10^{-3}
6	9.28×10^{-3}	9.74×10^{-3}
8	9.12×10^{-3}	9.62×10^{-3}

These results confirm that HI-SMURF preserves the expected $1/L$ -like error decay even for high-dimensional non-linear mappings and remains competitive with quantized MLP approximators under identical training conditions. The robustness of HI-SMURF across diverse multivariate operators under stochastic bitstream representations highlights its suitability for hardware-efficient approximation. Compared to lower-dimensional SMURF implementations that typically reach the 10^{-3} – 10^{-2} error scale at $L = 256$, HI-SMURF requires longer bitstreams (up to $L = 1024$) to achieve comparable accuracy, reflecting the increased variance amplification inherent in higher-dimensional function approximations.

In addition to varying bitstream length, we further examined the impact of tensor-train rank on approximation accuracy. Specifically, we evaluated HI-SMURF on the $M = 16$ softmax function under an idealized setting without bitstream-induced stochastic error. The results in Table IV show that as r increases from 2 to 8, the corresponding MAE decreases only slightly from 9.80×10^{-3} to 9.12×10^{-3} , while the RMSE drops from 1.04×10^{-2} to 9.62×10^{-3} . This modest improvement indicates that even low-rank decompositions (e.g., $r = 2$) already capture most of the functional behavior of the 16-variable softmax, and increasing r provides diminishing returns in accuracy. Consequently, HI-SMURF achieves near-optimal approximation fidelity with minimal rank, underscoring its efficiency for hardware-friendly deployment.

Bitstream correlation can significantly bias gate-level SC arithmetic (e.g., AND/XNOR-based operations) because multiple stochastic streams are directly combined. In contrast, HI-SMURF uses stochastic streams primarily to drive FSM state transitions, and produces outputs via a table-based mapping

from FSM states; thus, correlation mainly affects variance rather than strongly biasing the expected output. Consistent with this, our correlation ablation (fixed $L=128$) shows only minor RMSE variation when sweeping the target correlation coefficient from $\rho=1.0$ to $\rho=0.0$, indicating low sensitivity to input-stream correlation under the evaluated setting.

C. Performance of SMURF in an LDPC Decoder

LDPC algorithms and SC share a common focus on stochastic methods to process information. SC represents data and performs operations using random bitstreams, which can be advantageous for hardware implementations due to its simplicity and fault tolerance. LDPC decoding can leverage SC by using these bitstreams to perform iterative stochastic calculations, potentially reducing complexity and power consumption in decoding processes.

In LDPC decoding algorithms, variable nodes (VNs) and check nodes (CNs) play a crucial role in the iterative process. VNs represent the bits of the codeword and are responsible for updating their values based on messages received from the connected CNs. CNs ensure parity constraints are satisfied by processing messages from VNs and sending feedback to guide corrections. This collaborative message-passing between nodes iteratively refines the probability estimates, improving the accuracy of decoded data.

In the LDPC decoding algorithm, the channel information update formula is given by:

$$P_{AB} = \frac{P_A \cdot P_B}{P_A \cdot P_B + (1 - P_A) \cdot (1 - P_B)}, \quad (19)$$

where P_A represents the channel soft information, and P_B represents the soft information transmitted from the CN node connected to the current VN node. We compare the SMURF-based node update algorithm with the traditional Back-propagation (BP) algorithm and the Tree-based Feedback Message passing (TFM) algorithm. The comparison metric is the decoding process's BER, defined as:

$$\text{BER} = \frac{\text{Number of error bits}}{\text{Total number of transmitted bits}}. \quad (20)$$

The horizontal axis represents E_b/N_0 , where E_b is the energy per bit, and N_0 is the noise power spectral density.

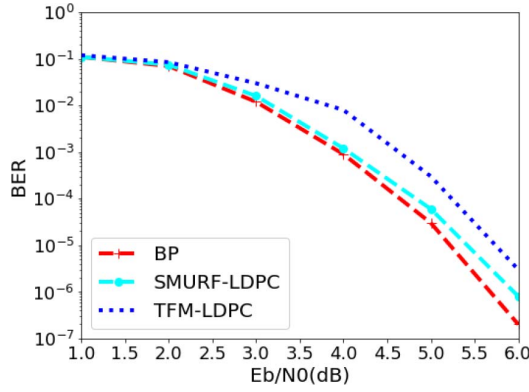


Fig. 9. The comparison of performance between SMURF and traditional LDPC decoding methods.

Fig. 9 compares the performance of our SMURF-LDPC approach with two traditional methods BP and TFM-LDPC, in terms of BER versus E_b/N_0 . It is evident that the SMURF-LDPC consistently outperforms the TFM-LDPC method, achieving a lower BER across all E_b/N_0 values. Compared to the BP method, SMURF-LDPC demonstrates comparable or superior performance, particularly at higher E_b/N_0 values, where it achieves significant BER reductions. These results highlight the robustness and effectiveness of the SMURF-LDPC algorithm, showcasing its ability to achieve better performance and reliability compared to traditional techniques. Consequently, SMURF-LDPC proves to be a competitive and reliable option for improving BER in LDPC-coded communication systems.

D. Performance of SMURF in Fourier Decomposition and Image Reconstruction

To evaluate SMURF's ability to approximate highly irregular image functions, we first decomposed target images into localized Fourier basis components. Directly training SMURF on pixel-wise grayscale values was found to be inefficient due to the non-smooth nature of high-frequency variations. Instead, by treating pixel coordinates (x_1, x_2) as input variables and approximating the image via Fourier basis functions, we transformed the reconstruction task into a set of smoother subproblems. Representative Fourier components with varying frequencies were fitted individually, as illustrated in Fig. 10. The results show that SMURF can accurately capture both low- and high-frequency Fourier bases, providing reliable building blocks for subsequent image synthesis.

After fitting the Fourier bases, the complete image was reconstructed by aggregating the individually approximated components. This two-stage process enables SMURF to handle both fine-grained details and global structural patterns. Fig. 11 reports the convergence behavior across different frequency terms, where lower-frequency functions converge faster while higher-frequency components require longer bitstreams and training epochs. Nevertheless, all tested functions eventually achieve stable approximation accuracy, confirming SMURF's scalability to complex spectral features.

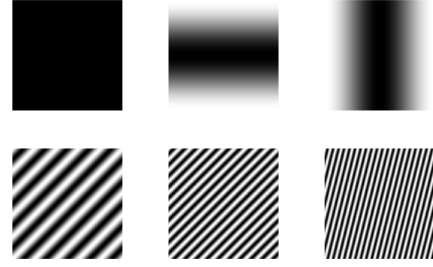


Fig. 10. Reconstructed Fourier basis images.

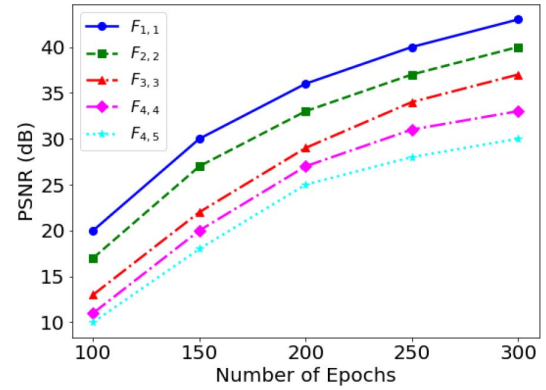


Fig. 11. Training convergence of SMURF on Fourier basis functions.

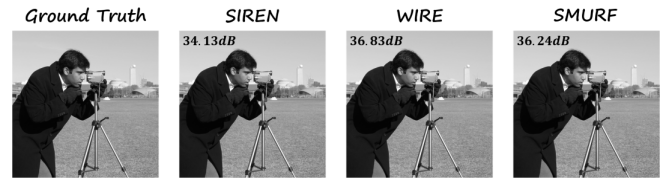


Fig. 12. Qualitative comparison between neural network-based INR, including SIREN [38] and WIRE [39], and our SMURF-based INR on **gray-scale image** reconstruction.

We then benchmarked SMURF against MLP-based INRs, including SIREN [38] and wavelet implicit neural representation (WIRE) [39]. As shown in Figs. 12, 13, and 14, SMURF achieves competitive or superior reconstruction quality across diverse scenarios. On a grayscale photograph, SMURF delivers a PSNR of 36.24dB, outperforming SIREN (34.13dB) and approaching WIRE (36.83dB). For computed tomography (CT) reconstruction, SMURF achieves 31.93dB, comparable to WIRE (32.37dB) and higher than SIREN (30.51dB), while preserving clinically relevant details. On color image reconstruction, SMURF attains 38.47dB, closely matching WIRE (38.53dB) and surpassing SIREN (38.26dB), faithfully capturing textures and subtle gradients. These results demonstrate that Fourier-decomposed training combined with stochastic FSM approximation provides an effective path for accurate image reconstruction, while maintaining substantial hardware efficiency advantages.

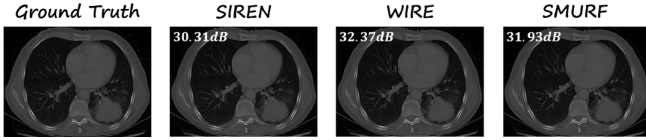


Fig. 13. Qualitative comparison between neural network-based INR, including SIREN [38] and WIRE [39], and our SMURF-based INR on CT image reconstruction.



Fig. 14. Qualitative comparison between neural network-based INR, including SIREN [38] and WIRE [39], and our SMURF-based INR on color image reconstruction.

E. Application of HI-SMURF in ViTs

To further evaluate the applicability of HI-SMURF in large-scale neural architectures, we integrated it into a lightweight ViT by replacing the softmax layer in the self-attention module. In particular, the compatibility scores produced by the query-key dot products were normalized using HI-SMURF rather than the conventional softmax function, while the remaining attention pipeline was left unchanged.

In a standard self-attention mechanism, the attention weights are computed as

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right), \quad (21)$$

where $\mathbf{Q} \in \mathbb{R}^{N_t \times d_k}$ and $\mathbf{K} \in \mathbb{R}^{N_t \times d_k}$ denote the query and key matrices, respectively; d_k is the key dimension used for scaling; and N_t is the number of input tokens. Each element a_{ij} in $\mathbf{A}$ represents the normalized compatibility score between the i th query vector $\mathbf{q}_i$ and the j th key vector $\mathbf{k}_j$. In the proposed HI-SMURF formulation, the variables $\mathbf{Q}$, $\mathbf{K}$, and d_k can all be treated as multivariate inputs to the HI-SMURF operator. Consequently, the normalization process does not require explicit precomputation of $\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$, since HI-SMURF directly approximates the nonlinear normalization function within its stochastic FSM framework.

This design enables HI-SMURF to serve as a drop-in replacement for the softmax function, thereby directly validating its effectiveness within Transformer-based models. The baseline model follows a vanilla Tiny ViT configuration. For the MNIST dataset, each 28×28 image was partitioned into 7×7 patches, resulting in 16 patch tokens. Each token was linearly projected into a 128-dimensional embedding space. The Transformer encoder consisted of 4 attention heads, with a hidden layer dimension of 512 (corresponding to 128×4). Training was performed for 10 epochs without batch normalization.

For CIFAR-10, images were normalized to mean values of (0.4914, 0.4822, 0.4465) and standard deviations of (0.2470, 0.2435, 0.2616). Each 32×32 image was divided into 4×4 patches, resulting in 16 patch tokens projected to 128-dimensional embeddings. The same encoder structure (4 heads, hidden dimension 512) was adopted, and training was conducted for 50 epochs with batch normalization enabled.

We further extended the evaluation to CIFAR-100 under the same training configuration. The experimental results are summarized in Table V. On MNIST, HI-SMURF slightly outperforms the baseline, achieving 98.4% validation accuracy compared to 98.2%. On CIFAR-10, HI-SMURF again yields a modest gain, reaching 76.9% versus 76.6% for the baseline. On the more challenging CIFAR-100 dataset, HI-SMURF maintains competitive convergence and surpasses the baseline, obtaining 47.2% validation accuracy compared to 46.9%. These results demonstrate that HI-SMURF can effectively replace the softmax operation in Transformer attention while delivering accuracy improvements across diverse datasets.

These results demonstrate that HI-SMURF can effectively replace the softmax operation in Transformer attention while delivering accuracy improvements across diverse datasets. Moreover, the consistent gains observed on MNIST, CIFAR-10, and CIFAR-100 suggest that HI-SMURF possesses strong generalization potential and can be readily extended to larger-scale datasets and deeper Transformer architectures.

F. Hardware Implementation

To further validate the efficacy of SMURF and HI-SMURF, we benchmarked its hardware metrics using an RTL-verified, synthesis-based evaluation flow. The RTL implementation was verified in ModelSim, and the hardware metrics were obtained by mapping the design to an SMIC 65 nm standard-cell technology library under a 400 MHz clock constraint. Unless otherwise stated, the reported “SMURF core” area/power are measured with two SNGs: one for runtime inputs and one for trained parameters feeding the MUX; the RNG macro ($1600 \mu\text{m}^2$) is included and dominates the SNG overhead. In our implementation, the RNG is realized by a lightweight tausURNG (combined Tausworthe PRNG), which generates one uniform random word per cycle to drive the SNG comparator (i.e., $\text{SNG} = \text{RNG} + \text{comparator}$); Box-Muller is only used when Gaussian/additive white Gaussian noise (AWGN) samples are required in auxiliary experiments. All hardware evaluations were performed under consistent precision settings: SMURF and HI-SMURF used 256-bit bitstreams, Bernstein used the same 256-bit configuration, and other baselines used 8-bit post-training quantization. The numerical accuracy (MAE/RMSE) has been reported in Subsections A and B. For higher-dimensional inputs, SNG cost can increase with parallel SNG replication; we use the two-SNG design as a practical latency-area trade-off. As an illustrative example, the bivariate Euclidean distance function was implemented, and a cubic Taylor-series polynomial was constructed to match SMURF’s approximation accuracy at a 256-bit bitstream length. SMURF employed two 4-state FSMs, consistent with the previous configuration.

TABLE V
PERFORMANCE OF HI-SMURF INTEGRATED INTO TINY ViT COMPARED WITH BASELINE SOFTMAX

Dataset	Epochs	Final Validation Loss	Final Validation Accuracy
MNIST (HI-SMURF)	10	0.08	98.4%
MNIST (Baseline)	10	0.09	98.2%
CIFAR-10 (HI-SMURF)	50	0.80	76.9%
CIFAR-10 (Baseline)	50	0.82	76.6%
CIFAR-100 (HI-SMURF)	50	1.50	47.2%
CIFAR-100 (Baseline)	50	1.52	46.9%

TABLE VI
HARDWARE METRICS AT SMIC 65 NM AND 400 MHz. DELAY IS THE EFFECTIVE PER-EVALUATION LATENCY;
ADP = AREA $\times$ DELAY. SMURF USES $L=64$ WITH $K=4$; BERNSTEIN USES $L=256$ WITH $K=4$. HI-SMURF
AND MLP VALUES ARE MEASURED UNDER THE SAME CONDITIONS FOR $M=16$

Methods	Area (μm^2)	Power (mW)	Delay (ns)	ADP ($\mu\text{m}^2\cdot\text{ns}$)
SMURF ($K=4$)	5.29×10^3	0.51	41.5	2.20×10^5
Taylor	3.29×10^4	3.53	15.8	5.20×10^5
LUT	2.38×10^5	0.10	8.3	1.98×10^6
TFM	1.17×10^4	1.17	37.2	4.37×10^5
Bernstein (SC, $L=256$, $K=4$)	7.95×10^3	0.68	166.8	1.33×10^6
CORDIC (Number of iterations=16, 16-bit)	1.61×10^4	1.60	104.6	1.67×10^6
HI-SMURF ($M=16$, $r=2$, $L=256$)	6.42×10^3	0.62	58.3	3.74×10^5
MLP (2×64 FC, 8-bit quant.)	1.85×10^5	4.92	22.4	4.14×10^6

SMURF consumes only 0.51 mW of power, primarily due to the RNG. Its total hardware area is $5.29 \times 10^3 \mu\text{m}^2$, where the RNG occupies approximately $1600 \mu\text{m}^2$, and the SMURF core and CPT-gate occupy $104.4 \mu\text{m}^2$ and $293.4 \mu\text{m}^2$, respectively. The Taylor approximation, implemented as a 4-stage pipeline with 16-bit quantization, consumes 3.53 mW and occupies $3.29 \times 10^4 \mu\text{m}^2$. Although LUTs exhibit low power consumption (0.10 mW), they require a large area of $2.38 \times 10^5 \mu\text{m}^2$. We also include results for the TFM-based VN design for LDPC decoding, which consumes 1.17 mW and occupies $1.17 \times 10^4 \mu\text{m}^2$. Finally, we benchmark the Bernstein stochastic approximation, which uses a bitstream length of 256, resulting in larger latency but significantly smaller area than LUT-based methods. Bernstein requires approximately 282 logic gates and 1128 transistors, slightly higher than SMURF but still orders of magnitude lower than traditional Taylor or LUT implementations.

Table VI summarizes the hardware metrics, including effective delay and ADP. Thanks to parallelization ($K=4$), SMURF achieves a delay of 41.5 ns and an ADP of $2.20 \times 10^5 \mu\text{m}^2\cdot\text{ns}$, which is less than half of Taylor's $5.20 \times 10^5 \mu\text{m}^2\cdot\text{ns}$ and well below LUT-based designs ($1.98 \times 10^6 \mu\text{m}^2\cdot\text{ns}$). Bernstein's SC approximation has a delay of 166.8 ns due to its longer bitstream, resulting in an ADP of $1.33 \times 10^6 \mu\text{m}^2\cdot\text{ns}$, still significantly better than LUT in terms of area overhead. TFM

VNs, although smaller than LUT, incur a 37.2 ns delay and a $4.37 \times 10^5 \mu\text{m}^2\cdot\text{ns}$ ADP.

While SC incurs a window-based per-evaluation latency that scales with the bitstream length L , system throughput can be preserved via spatial parallelism and replication, which is well matched to iterative graph workloads such as LDPC decoding. In our implementation, the effective delay in Table VI reflects this co-design: with parallelization ($K=4$), SMURF evaluates an output in approximately L/K cycles (plus minor control overhead), yielding 41.5 ns at 400 MHz for $L=64$. In LDPC, many variable/check nodes are updated concurrently, and thus the throughput is largely determined by the number of processing elements and the scheduling; SMURF/HI-SMURF's favorable ADP makes throughput scaling via compact replication more efficient than using arithmetic-heavy fixed-point approximators.

SMURF's area is only 16.1% of Taylor's and 2.22% of LUT's, while consuming just 14.4% of Taylor's power. Although LUTs exhibit lower power usage, their massive area overhead makes them impractical for dense designs. Compared to TFM, SMURF reduces ADP by 49.7%, demonstrating its efficiency for LDPC-like variable nodes. CORDIC (16 iterations, 16-bit) has a moderate area footprint ($1.61 \times 10^4 \mu\text{m}^2$), still far smaller than LUT/MLP, but its longer effective latency (104.6 ns) yields a higher ADP ($1.67 \times 10^6 \mu\text{m}^2\cdot\text{ns}$) than

SMURF, highlighting the throughput-latency trade-off of iterative nonlinear evaluation. Bernstein SC, despite its longer latency, remains area-efficient compared to LUT and achieves an ADP ($1.33 \times 10^6 \mu\text{m}^2\cdot\text{ns}$) lower than CORDIC, while still being higher than SMURF due to the bitstream-induced delay.

For the 16-variable experiments, HI-SMURF preserves accuracy while containing hardware cost growth through tensor-train factorization. On top of SMURF, HI-SMURF adds only a small number of extra AND gates for the TT multiplications, a cost negligible compared with the exponential parameter growth avoided by the factorization. Compared with its lower-dimensional counterpart SMURF, HI-SMURF shows only a moderate area increase (about 21% more in our FPGA synthesis) when scaling from low-dimensional functions to the 16-variable setting, yet it maintains stable latency and power characteristics. This demonstrates that HI-SMURF effectively mitigates the exponential parameter growth that would otherwise occur in direct SMURF implementations as dimensionality increases. The quantized MLP used for the same 16-variable functions, although capable of achieving competitive numerical accuracy at high precision, exhibits substantially larger area and power requirements relative to HI-SMURF under identical conditions. These findings confirm that HI-SMURF offers a hardware-friendly path to high-dimensional function approximation, whereas MLP-based approaches remain costly in resource-constrained accelerators.

Fig. 15 highlights that SMURF requires only 188 logic gates and 752 transistors, which is two to three orders of magnitude fewer than conventional approaches (e.g., 13,900 gates for Taylor expansion or 142,700 transistors for LUT). Bernstein requires 282 logic gates and 1,128 transistors, remaining far more efficient than Taylor or LUT. The quantized MLP design, in contrast, incurs a dramatic increase in hardware complexity, consuming roughly 6,000 logic gates and 240,000 transistors. HI-SMURF, which tensorizes the parameter table for 16-variable functions, modestly increases hardware over SMURF but still uses only 220 logic gates and 880 transistors, keeping its footprint extremely small compared to MLP. These results underscore SMURF's and HI-SMURF's ability to replace arithmetic-heavy computation with probabilistic FSM state transitions, enabling ultra-low-power nonlinear processing while maintaining accuracy, which is an advantage that makes them highly suitable for resource-constrained edge devices.

G. Additional Remarks

The proposed SMURF architecture provides accurate and efficient approximation of diverse nonlinear functions through an SC-based FSM design, achieving substantial reductions in area and power compared with conventional methods such as Taylor and LUT implementations. Its parameterizable θ -gates enable flexible reconfiguration for different functions using the same hardware. Building on this foundation, HI-SMURF extends the approach to high-dimensional operators with only marginal hardware overhead, retaining SMURF's compactness and energy efficiency. Together, SMURF and HI-SMURF offer a unified, reconfigurable, and resource-efficient framework

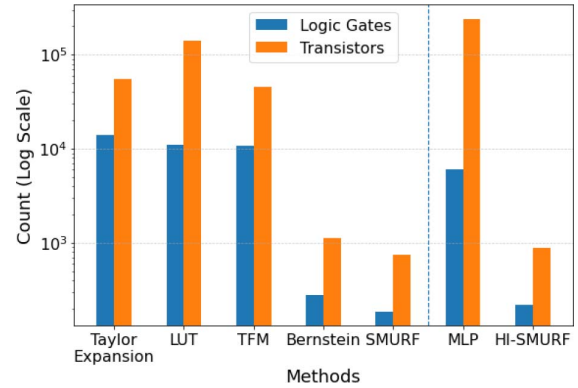


Fig. 15. Hardware resource comparison of SMURF, Bernstein, and traditional computing methods in terms of logic gate and transistor count.

for implementing nonlinear computations in AI and signal-processing accelerators.

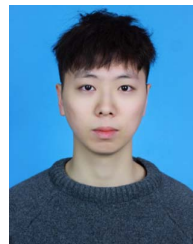
V. CONCLUSION

In this paper, we have presented SMURF, a stochastic computing architecture based on finite-state machines for efficient approximation of multivariate nonlinear functions, together with its high-dimensional extension, HI-SMURF. We have established a unified analytical framework for steady-state behavior, detailed hardware implementations, and tensor-train-based parameterization to alleviate state-space explosion. Extensive evaluations across representative nonlinear functions and practical applications, including LDPC decoding and Vision Transformers, demonstrate that both SMURF and HI-SMURF achieve competitive numerical accuracy while consistently delivering superior hardware efficiency in terms of area, power, and area-delay product compared with conventional approximators such as Taylor, LUT, Bernstein, and quantized MLPs. These results highlight the scalability of HI-SMURF for high-dimensional operators and its potential as a flexible and reconfigurable coprocessor for resource-constrained edge-AI workloads.

REFERENCES

- [1] X.-D. Zhang, *Modern Signal Processing*. Berlin, Germany: Walter de Gruyter GmbH & Co KG, 2022.
- [2] J. A. Zhang et al., "An overview of signal processing techniques for joint communication and radar sensing," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 6, pp. 1295–1315, Nov. 2021.
- [3] J. G. Proakis and M. Salehi, *Digital Communications*, vol. 4. New York, NY, USA: McGraw-Hill, 2001.
- [4] C. Tse and F. Lau, *Chaos-Based Digital Communication Systems. Operating Principles, Analysis Methods and Performance Evaluation*. Berlin, Germany: Springer Verlag, 2003, pp. 45–56.
- [5] J. Xie and Y. Lakys, "Application of nonlinear fractional differential equations in computer artificial intelligence algorithms," *Appl. Math. Nonlinear Sci.*, vol. 8, no. 1, pp. 1145–1154, 2023.
- [6] A. D. Jagtap and G. E. Karniadakis, "How important are activation functions in regression and classification? A survey, performance comparison, and future directions," *J. Mach. Learn. Model. Comput.*, vol. 4, no. 1, pp. 21–75, 2023.
- [7] J. Lederer, "Activation functions in artificial neural networks: A systematic overview," 2021, *arXiv:2101.09957*.
- [8] J. Cheng, H. Yan, J. H. Park, and G. Zong, "Output-feedback control for fuzzy singularly perturbed systems: A nonhomogeneous stochastic

- communication protocol approach,” *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 76–87, Jan. 2023.
- [9] Y. Fang, Y. Bu, P. Chen, F. C. M. Lau, and S. A. Otaibi, “Irregular-mapped protograph LDPC-coded modulation: A bandwidth-efficient solution for 6G-enabled mobile networks,” *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2060–2073, Feb. 2023.
 - [10] Q. Liu, L. Xiao, J. Yang, and Z. Wei, “CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification,” *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.
 - [11] S. T. Thomdapu, H. Vardhan, and K. Rajawat, “Stochastic compositional gradient descent under compositional constraints,” *IEEE Trans. Signal Process.*, vol. 71, pp. 1115–1127, 2023.
 - [12] X. Deng et al., “A kernelized support tensor-ring machine for high-dimensional data classification,” in *Proc. Int. Conf. Electron. Inf. Technol. Smart Agriculture (ICEITSA)*, 2021, pp. 159–165.
 - [13] J. Gao, F. Li, B. Wang, and H. Liang, “Unsupervised nonlinear adaptive manifold learning for global and local information,” *Tsinghua Sci. Technol.*, vol. 26, no. 2, pp. 163–171, 2021.
 - [14] G. Menghani, “Efficient deep learning: A survey on making deep learning models smaller, faster, and better,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–37, 2023.
 - [15] D. Ghimire, D. Kil, and S-h Kim, “A survey on efficient convolutional neural networks and hardware acceleration,” *Electronics*, vol. 11, no. 6, 2022, Art. no. 945.
 - [16] K. Sharifani and M. Amini, “Machine learning and deep learning: A review of methods and applications,” *World Inf. Technol. Eng. J.*, vol. 10, no. 7, pp. 3897–3904, 2023.
 - [17] K. S. Zaman, M. B. I. Reaz, S. H. Md Ali, A. A. A. Bakar, and M. E. H. Chowdhury, “Custom hardware architectures for deep learning on portable devices: A review,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 11, pp. 6068–6088, Nov. 2022.
 - [18] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, “Transformers in vision: A survey,” *ACM Comput. Surv. (CSUR)*, vol. 54, no. 10s, pp. 1–41, 2022.
 - [19] P. Dong et al., “HeatViT: Hardware-efficient adaptive token pruning for vision transformers,” in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 442–455.
 - [20] P. Veličković, “Everything is connected: Graph neural networks,” *Current Opin. Struct. Biol.*, vol. 79, Apr. 2023, Art. no. 102538.
 - [21] A. Vaswani et al., “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, vol. 30, pp. 5998–6008.
 - [22] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” 2020, *arXiv:2010.11929*.
 - [23] C. Yang and Y. Guo, “Numerical solution of nonlinear Fredholm integral equations of second kind by using hybrid of block-pulse functions and Taylor series,” in *Proc. IEEE Int. Conf. Prog. Inform. Comput.*, 2010, vol. 1, pp. 453–455.
 - [24] MalayK. Ganai and F. Ivančić, “Efficient decision procedure for nonlinear arithmetic constraints using CORDIC,” in *Proc. Formal Methods Comput.-Aided Des.*, 2009, pp. 61–68.
 - [25] P. Behrooz, *Computer Arithmetic: Algorithms and Hardware Designs*, vol. 19. London, U.K.: Oxford Univ. Press, 2000, pp. 512583–512585.
 - [26] R. Schramm, “On piecewise linear functions and piecewise linear equations,” *Math. Oper. Res.*, vol. 5, no. 4, pp. 510–522, 1980.
 - [27] P. Blackburn and M. Marx, “Constructive interpolation in hybrid logic,” *J. Symbolic Logic*, vol. 68, no. 2, pp. 463–480, 2003.
 - [28] Tsu-Tian Lee, Jin-Tsong Jeng, “The Chebyshev-polynomials-based unified model neural networks for function approximation,” *IEEE Trans. Syst., Man, Cybern. B. Cybern.*, vol. 28, no. 6, pp. 925–935, Dec. 1998.
 - [29] B. Li, P. Gu, Y. Shan, Y. Wang, Y. Chen, and H. Yang, “RRAM-based analog approximate computing,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 12, pp. 1905–1917, Dec. 2015.
 - [30] B. R. Gaines, *Stochastic Computing Systems*. Boston, MA, USA: Springer US, 1969, pp. 37–172.
 - [31] B. D. Brown and H. C. Card, “Stochastic neural computation. I. Computational elements,” *IEEE Trans. Comput.*, vol. 50, no. 9, pp. 891–905, Sep. 2001.
 - [32] X. Feng, K. Hu, and K. Han, “MM-FSM: A high-efficiency general nonlinear function generator for stochastic computation,” *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 1998–2009, Sep. 2022.
 - [33] W. Qian, Marc, and D. Riedel, “The synthesis of robust polynomial arithmetic with stochastic logic,” in *Proc. 45th Annu. Des. Automat. Conf.*, 2008, pp. 648–653.
 - [34] R. Hojabr et al., “SkipPyNN: An embedded stochastic-computing accelerator for convolutional neural networks,” in *Proc. 56th ACM/IEEE Des. Autom. Conf. (DAC)*, 2019, pp. 1–6.
 - [35] E. M. Jonas “Stochastic architectures for probabilistic computation,” Ph.D. thesis, MIT, Cambridge, MA, USA, 2014.
 - [36] A. A. Markov, “Extension of the limit theorems of probability theory to a sum of variables connected in a chain,” S. Petelin, Transl., in *Dynamic Probabilistic Systems*, vol. 1, R. A. Howard, Ed., New York, NY, USA: Wiley, 1971, pp. 552–576.
 - [37] Ivan, Voseledets, “Tensor-train decomposition,” *SIAM J. Scientific Comput.*, vol. 33, no. 5, pp. 2295–2317, 2011.
 - [38] V. Sitzmann et al., “Implicit neural representations with periodic activation functions,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, vol. 33, pp. 7462–7473.
 - [39] V. Saragadam et al., “WIRE: Wavelet implicit neural representations,” in *Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit.*, 2023, pp. 18507–18516.



stochastic computation to the fundamental architecture and algorithms of artificial intelligence.



computing paradigms.



applications in brain-computer interfaces and artificial intelligence.

Xincheng Feng received the bachelor's degree in mathematics from Beihang University (Beijing University of Aeronautics and Astronautics), in 2019, and the master's degree in information and communication engineering from the National Key Laboratory of Wireless Communication, University of Electronic Science and Technology of China, in 2022. He is currently working toward the Ph.D. degree with the Department of Electrical and Electronic Engineering, The University of Hong Kong. His research interests include the application of

Wenying Zhou received the B.S. degree in electronic science and technology from Tianjin University, China, in 2019, the M.S. degree in computer engineering from the Northwestern University, Evanston, IL, USA, in 2021, and the Ph.D. degree in electrical and electronic engineering from the University of Hong Kong (HKU), Hong Kong, in 2025. His research interests include efficient data representation such as implicit neural representations and model compression, particularly quantization and pruning. He is also interested in emerging efficient

Taiqiang Wu received the B.S. degree in automation and the M.S. degree in electronic and information engineering (artificial intelligence) from Tsinghua University, China, in 2020 and 2023, respectively. He is currently a third-year Ph.D. student with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong SAR. His research interests include model compression and efficient fine-tuning for large language models.

Zhengwu Liu (Member, IEEE) received the B.E. degree in electronic engineering from the University of Electronic Science and Technology of China in 2018, and the Ph.D. degree in electrical engineering from Tsinghua University in 2023. He is currently a Research Assistant Professor with the Department of Electrical and Electronic Engineering, The University of Hong Kong. He has coauthored over 20 peer-reviewed papers in journals and conferences. His research interests include neuromorphic computing, compute-in-memory architectures, and their



Meng Li received the B.S. degree from Peking University, China, in 2013, and the Ph.D. degree from The University of Texas at Austin, USA, in 2018. He is a Tenure-Track Assistant Professor with Peking University, jointly affiliated with the Institute for Artificial Intelligence and the School of Integrated Circuits. Before joining Peking University, he was a Staff Research Scientist and Tech Lead with Facebook On-Device AI Team, working on efficient AI algorithms and hardware for AR/VR devices. He has published over 80 articles. His research interests include efficient and secure multimodal AI acceleration algorithms and hardware. He received Best Paper Awards from HOST 2017 and GLSVLSI 2018, the CCF Integrated Circuits Early Career Award, the EDAA Outstanding Dissertation Award, and First Place in the ACM Student Research Competition Grand Final (Graduate Category).



Ngai Wong (Senior Member, IEEE) received the B.Eng. and Ph.D. degrees in electrical and electronic engineering from The University of Hong Kong (HKU). He is currently an Associate Professor with the Department of Electrical and Electronic Engineering, HKU. He was a Visiting Scholar with Purdue University, USA, in 2003. He served as an Associate Editor of IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS (2014–2022) and as track chair or TPC member for DAC, ICCAD, and ASP-DAC. He co-founded the IEEE Council on EDA (CEDA) Hong Kong Chapter in 2016 and served as the Chair in 2018/19. His research interests include compact neural network design, compute-in-memory AI chips, EDA, and tensor algebra.